

ODD系列（一）：AI原生软件工程中的产出物合法性基础

统一确权引用 / Canonical Reference (Zenodo DOI)

中文（推荐引用）：Yi Fu. 《输出驱动开发：AI辅助软件工程的范式转变》. Zenodo, 2026. DOI: 10.5281/zenodo.18207648. (<https://doi.org/10.5281/zenodo.18207648>)

English (recommended citation): Yi Fu. Output-Driven Development: A Paradigm Shift in AI-Assisted Software Engineering. Zenodo, 2026. DOI:

10.5281/zenodo.18207648. (<https://doi.org/10.5281/zenodo.18207648>)

Record: <https://zenodo.org/records/18207648>

```
@misc{odd_zenodo_18207648,
  author    = {Yi Fu},
  title     = {Output-Driven Development: A Paradigm Shift in AI-Assisted
Software Engineering},
  year      = {2026},
  publisher = {Zenodo},
  doi       = {10.5281/zenodo.18207648},
  url       = {https://doi.org/10.5281/zenodo.18207648}
}
```

作者: Yi Fu . (ODDFounder, fuyi.it@live.cn)

日期: 2026-01-12

版本: v7.1 (中文版)

摘要

大型语言模型（LLM）的代码生成能力在2023-2025年间经历了质的飞跃，创造了一个前所未有的工程困境：**AI生成代码的速度远超人类审查的速度**。传统软件开发方法论（敏捷、TDD、BDD等）都假设“人工代码审查”是质量保证的最后防线——这一假设在AI时代已经失效。

本文提出输出驱动开发（ODD），一种专为AI辅助软件工程设计的新范式。ODD的核心创新包括：

1. 产出物中心：软件开发的目标不是生成代码，而是生成满足人类需求的产出物
2. 契约驱动：使用精确契约定义产出物规格，使需求可量化、可验证、可测试
3. 变异测试信任：用变异测试替代人工审查作为信任基础
4. 封存保护：通过可审计、可追溯的版本管理保护已验证的代码

我们认为，ODD不仅是方法论创新，更是软件开发生产关系的系统性重构：实现了脑力劳动与执行劳动的历史性分离，推动软件行业从"手工作坊模式"迈向"智能工厂模式"。

关键词：ODD、输出驱动开发、产出物、契约、AI辅助开发、软件工程、范式转变、变异测试

1. 引言

1.1 背景：破碎的控制面

大型语言模型（LLM）的最新进展从根本上改变了软件的生产方式[6, 7]。

Copilot、ChatGPT和Claude等代码生成系统不再是辅助工具；它们越来越多地充当可执行产出物的主要生产者。

然而，虽然生成能力快速进步，软件工程的控制结构却没有相应演进。

传统软件工程方法论——包括敏捷、DevOps、测试驱动开发（TDD）和模型驱动工程（MDE）——隐含地假设：

- 源代码由人类编写，
- 对行为的推理通过代码检查来中介，
- 责任可以通过作者身份和提交历史来追溯。

这些假设在AI辅助生产中不再可靠成立。

因此，现代软件系统越来越呈现出一种悖论状态：

代码丰富，但责任分散。
自动化强大，但控制脆弱。

1.2 核心问题：责任锚点的丧失

AI辅助软件生成引入的核心挑战不仅仅是正确性，而是责任锚点的侵蚀。

在传统工作流中，责任隐含地附着于：

- 代码的人类作者，
- 审查过程，
- 或模块的所有权。

然而，在AI辅助工作流中：

- 代码可能被生成、修改、再生成多次，

- 生成路径不透明且非确定性，
- 多个代理（人类和AI）可能异步贡献。

在这种条件下，"谁负责？"这个问题在结构上变得无法定义。

现有的应对措施主要试图通过以下方式恢复控制：

- 改进提示工程[8]，
- 添加更多测试，
- 约束模型或沙箱执行。

虽然有用，但这些方法在*代码中心*范式内运作，因此无法解决更深层的结构性转变。

1.3 为什么代码中心的控制不再足够

代码中心的方法论假设：

- 理解代码意味着理解系统行为，
- 审查代码意味着验证意图，
- 维护代码意味着随时间保持正确性。

在AI辅助开发中，这些假设因三个原因而退化：

1. 人类不可读性

生成的代码可能语法有效、功能正确，但认知上不透明。

2. 短暂的实现

代码变得可丢弃：重新生成通常比维护更便宜。

3. 意图与实现的解耦

表达意图的实体（人类）不再是产生实现的实体（AI）。

因此，代码不能再作为控制、验证或责任的主要场所。

1.4 重构问题：从"代码如何编写？"到"产出什么？"

本文认为，适当的应对不是进一步优化代码生成，而是重构软件工程本身的单元。

不再问：

- 代码应该如何编写？
- AI如何生成更好的代码？

我们提议问：

- 什么输出是可接受的？
- 这些输出如何被验证、审计和拥有？

- 当实现是短暂的时，责任在哪里？

这种转变激发了一种范式，其中产出的输出产出物，而非代码或生成过程，成为控制的主要对象。

定义：ODD中的产出物是软件开发过程中产生的可验证输出，满足特定人类需求并具有使用价值。产出物是软件开发的真正目标——而非代码本身。

1.5 输出驱动开发（ODD）：范式转变

为解决上述挑战，我们引入输出驱动开发（ODD），一种面向AI辅助软件工程的产出物中心方法论。

ODD建立在三个核心原则之上：

1. 输出作为一等实体
产出物被明确指定、验证和审计。
2. 生成前的契约
在任何AI执行之前定义可接受的输出空间。
3. 通过仲裁的责任
人类被定位为契约审查者和最终仲裁者，而非日常实现者。

ODD不是试图消除人类参与，而是将人类注意力集中在不可约减的决策点上，同时将日常生成和验证委托给AI系统。

1.6 贡献

本文做出以下贡献：

1. 概念重构
我们正式定义输出驱动开发作为AI原生软件工程的替代控制范式。
2. 基础公理
我们阐明区分ODD与代码中心方法论的核心公理。
3. 责任重新分配
我们重新定义人类在AI辅助工作流中的角色为产出物和契约的审查者和仲裁者。
4. 未来验证的基础
我们建立一个框架，在此基础上可以构建经验验证、工具和治理机制。

1.7 范围和局限性

本文聚焦于基础定义和概念结构。

我们不声称：

- 在所有领域的经验优越性，
- 对探索性或创造性编程的即时适用性，
- 替代现有工程实践。

相反，本工作提供了一个结构性视角，用于推理AI辅助软件生产中的责任、控制和合法性。

经验评估和系统级验证在ODD系列的后续论文中讨论。

本工作不旨在最大化自动化或替代人类智能。相反，它引入结构性约束，以确保在AI辅助生产下，责任、可审计性和人类仲裁保持完整和可扩展。

免责声明：本文建立概念框架和可测试假设。使用我们参考实现（Progee）的生产数据进行经验验证计划作为近期未来工作。本工作作为预印本发布，尚未经过同行评审。

1.8 论文组织

本文其余部分组织如下：

- 第2节正式定义输出驱动开发及其核心公理。
- 第3节将ODD与现有软件工程范式进行对比。
- 第4节讨论对验证、责任和治理的影响。
- 第5节概述局限性和未来工作方向。

2. 核心概念

2.1 产出物：软件开发的真正目标

2.1.1 什么是产出物？

定义：产出物是软件开发过程中产生的可验证输出，能够满足特定人类需求。

产出物的本质定义

产出物 = 可验证的输出 + 满足特定需求 + 具有使用价值

产出物的三个核心属性：

1. 可验证性

- 正确性可以通过测试确认
- 合规性可以对照契约检查

2. 需求满足性

- 对应明确的人类需求
- 解决具体的业务问题

3. 使用价值

- 可以被用户直接使用
- 产生实际效用

2.1.2 产出物与人类需求的关系

哲学洞见：人类从根本上需要的是结果，而非过程。

类比1：香肠与百元大钞

想象这个场景：你饿了，想吃香肠。你有100美元。

香肠交易的本质

[\$100] → [香肠]
(输入) 你关心中间发生了什么吗? (产出物)

肉厂的机器？厨师的烹饪？收银员？

答案：你完全不关心。你只想要香肠。
如果有个魔法盒子能直接把钱变成香肠，
你会毫不犹豫地使用它。

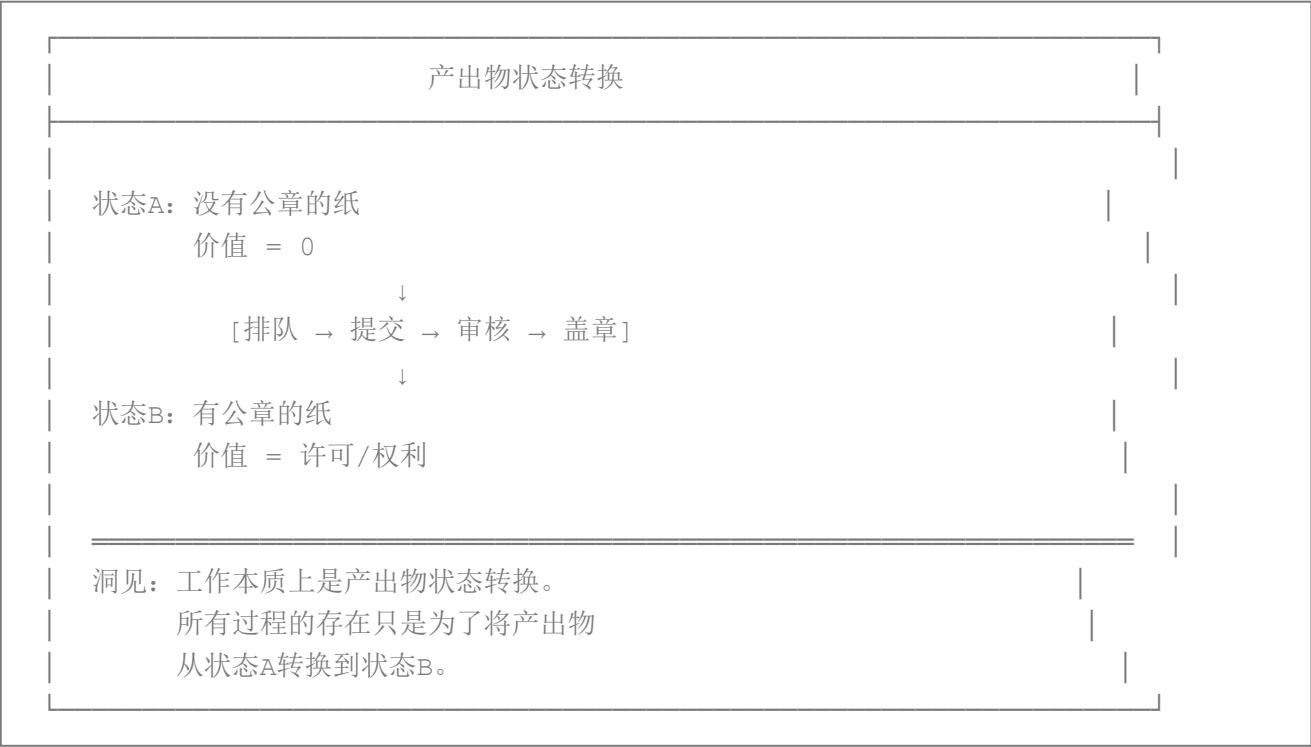
洞见：过程是开发者的自我陶醉；
产出物才是客户的真正需求。

类比2：公章与白纸

你去政府办事处。你手里拿着一张写满文字的纸（申请书）。

你的目的是什么？不是"排队"，不是"和办事员交谈"，不是"看着他们在纸上盖章"。

你唯一的目的是：在这张纸上得到一个红色的公章。



软件领域映射：

客户给你100万美元做一个电商系统。他们不关心你用Java还是Go，微服务还是单体。他们只关心：

- 用户能下单吗？
- 支付能成功吗？
- 物流能追踪吗？

这些就是产出物——可以使用并创造价值的东西。

2.1.3 产出物分类体系

ODD将软件开发产出物分为698种类型，每种都有明确的定义、模板和验收标准。

698种产出物类型分类（摘录）			
01. 功能产出物（约300种）			
01.01	API端点	01.02	业务服务
01.03	数据处理器	01.04	用户界面
01.05	后台任务	01.06	集成适配器
02. 验证产出物（约150种）			
02.01	单元测试	02.02	集成测试
02.03	端到端测试	02.04	性能测试

02.05 安全测试	02.06 变异测试配置
03. 配置产出物（约100种）	
03.01 应用配置	03.02 环境配置
03.03 构建配置	03.04 部署配置
03.05 监控配置	03.06 安全配置
04. 文档产出物（约80种）	
04.01 API文档	04.02 架构文档
04.03 用户手册	04.04 运维手册
04.05 变更日志	04.06 决策记录
05. 契约产出物（约68种）	
05.01 功能契约	05.02 API契约
05.03 数据契约	05.04 性能契约
05.05 安全契约	05.06 集成契约

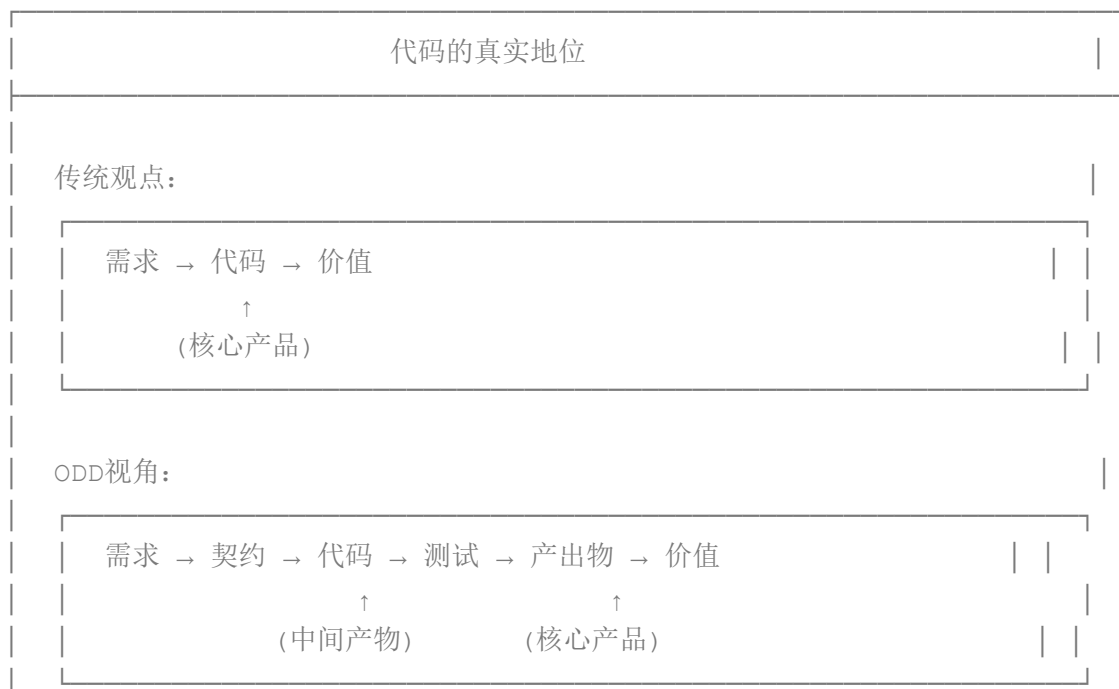
为什么是698种？

- 更细的分类 = AI更准确的理解
- 每种类型有专用模板，减少AI"创造性"
- AI理解这些类别；人类只需定义一次
- 支持自动化验收和质量度量

2.2 为什么生成产出物而非代码？

2.2.1 代码的真实地位

传统思维认为程序员的工作是"写代码"，代码是程序员的"作品"。但这是一个历史性误解。



类比：香肠工厂

- 香肠 = 产出物（客户想要的）
- 绞肉机、搅拌机 = 代码（生产工具）
- 废水、边角料 = 代码副产品（不可避免的输出）

当你买香肠时，你不会问绞肉机的型号。

当客户买软件时，他们不应该关心你的代码有多"优雅"。

2.2.2 传统编程思维的三大幻觉

传统编程思维的三大幻觉

幻觉1：写代码本身就是价值

→ 真相：代码只是达到目的的手段

→ 类比：你沉迷于磨刀，
但客户只想要那道菜

幻觉2：代码是资产

→ 真相：代码是负债（越多 = 越难维护，
越难理解，越多bug）

→ 类比：工厂机器越多 = 维护成本越高

幻觉3：代码质量 = 软件质量

→ 真相：产出物正确性 = 软件质量

→ 类比：香肠好不好吃不取决于
绞肉机有多干净

程序员常见的"自我陶醉"：

- 刀工（代码风格） → 客户只想要那道菜
- 摆盘（架构） → 客户只关心味道
- 厨具（框架） → 客户只想吃饱

这些对专业厨师很重要，但对饥饿的客户来说，
它们是次要的。

2.2.3 为什么ODD聚焦于产出物

为什么ODD聚焦于产出物
原因1：产出物是可验证的
• 代码是否"优雅"是主观的 • 产出物是否"正确"是客观事实 • 我们可以测试产出物；我们无法测试"代码美学"
原因2：产出物对应人类需求
• 代码对应"实现方式" • 产出物对应"用户故事" • 客户验收产出物，而非代码行数
原因3：产出物可以被封存和保护
• 代码可能被意外修改或覆盖 • 一旦验收，产出物可以被封存 • 封存的产出物是稳定的"积木"
原因4：AI擅长生成代码，但验收需要人类
• AI可以快速生成大量代码 • 但AI无法判断"这是用户想要的吗？" • 产出物定义和验收是人类的工作 • ODD：人类专注于"定义什么"；AI专注于"如何实现"

2.2.4 范式转变：从"生成代码"到"生成产出物"

范式转变：从"代码"到"产出物"
传统AI辅助开发（如GitHub Copilot）：
人类编写 → AI补全 → 人类审查 → 人类测试 ↑ ↑ ↑ (人类主导) (AI辅助) (瓶颈！)
问题：AI生成速度是人类的100倍， 但审查速度仍然是人类速度
结果：AI越快 = 人类越忙 = 瓶颈越大

ODD方式：

人类定义契约 → AI生成 → 系统验证 → 自动封存

↑
(人类主导)

↑
(无人人类瓶颈)

关键转变：

- 人类不再审查代码——他们定义契约
- 系统通过变异测试验证，而非人眼
- 产出物是核心；代码只是生产它们的手段

3. 输出驱动开发的语境定位

3.1 为什么ODD不是渐进式改进

乍一看，输出驱动开发（ODD）可能看起来是现有软件工程实践的渐进式改进。然而，这种解读从根本上误解了它的意图。

ODD不是优化软件如何生产。

它重新定义了什么构成工程控制的对象。

本节将ODD与已建立的范式进行对比，并阐明为什么ODD不能被简化为或归入其中任何一个。

3.2 ODD vs. 敏捷和DevOps

敏捷和DevOps强调：

- 快速迭代，
- 持续反馈，
- 开发与部署的紧密对齐。

这些方法论假设：

- 代码变更可追溯到人类决策，
- 系统行为增量演进，

- 责任可以从过程参与中推断。

ODD在根本上有所不同。

维度	敏捷 / DevOps	ODD
主要控制单元	代码与过程	输出产出物
变更成本	增量管理	通常重新生成
责任	按角色分布	通过仲裁锚定
假设	人类编写代码	AI生成产出物

ODD与敏捷交付管道兼容，但与其控制假设正交。

3.3 ODD vs. 测试驱动开发（TDD）

测试驱动开发通过以下方式强制正确性：

- 在实现之前编写测试，
- 将测试视为可执行规格。

虽然TDD提升了测试的地位，但它仍然从根本上是代码中心的：

- 测试验证实现，
- 测试覆盖率近似行为置信度，
- 责任通过测试作者身份来中介。

相比之下，ODD：

- 通过契约指定 *可接受的输出空间*，
- 直接评估产出物，
- 使用测试、变异分析和对抗验证作为支持机制——而非主要规格。

方面	TDD	ODD
规格形式	测试	产出物契约
验证目标	实现行为	输出有效性
失败解释	测试失败	产出物违反契约

因此，ODD不是替代TDD，而是将其重新定位为从属验证层。

3.4 ODD vs. 模型驱动工程（MDE）

模型驱动工程提倡：

- 抽象模型作为主要产出物，
- 从模型生成代码，
- 模型与实现之间的一致性。

尽管表面相似，ODD在两个关键方面有所不同：

1. 认识论地位
- MDE模型旨在表示系统结构。
- ODD契约定义可接受的结果，而不声称表示完整性。
2. 责任场所
- 在MDE中，正确性源于模型保真度。
- 在ODD中，正确性源于产出物验证和仲裁。

维度	MDE	ODD
主要抽象	结构模型	输出契约
失败模式	模型-代码偏离	产出物违规
人类角色	模型作者	契约仲裁者

ODD明确避免将契约视为完整的系统模型。

3.5 ODD vs. 提示工程和LLM导向实践

最近的AI辅助 workflow 强调：

- 提示工程，
- 思维链控制，
- 微调或检索增强。

这些方法专注于塑造生成行为。

ODD故意将自己与生成机制解耦：

在ODD中，产出物的质量必须是可辩护的，无论它是如何生成的。

这一设计选择有两个后果：

- 1. ODD保持模型无关性。
- 2. 责任不会因生成复杂性而稀释。

提示工程可能改善输出，但它不能替代产出物级别的问责。

3.6 ODD vs. 人在回路（HITL）系统

人在回路系统强调：

- 频繁的人类干预，
- 生成过程中的监督，
- 交互式纠正。

ODD重构了这种关系。

维度	HITL	ODD
人类参与	持续	有条件
干预触发	过程阶段	不可约减的分歧
认知负荷	高	集中

ODD不是"人在回路外"。
它是人在决策点。

3.7 总结：正交性，而非替代

ODD不寻求替代现有方法论。
相反，它引入了一个与代码中心实践正交的新控制轴。

现有方法优化生产。
ODD治理合法性。

这一区别解释了为什么随着AI系统扩展、实现变得短暂、责任必须保持明确，ODD变得越来越相关。

3.8 过渡到第4节

在将ODD与现有范式进行对比之后，下一节将考察产出物中心控制模型的影响，涉及：

- 验证策略，
- 责任分配，

- 以及AI原生软件系统中的治理。

4. 产出物中心控制模型的影响

4.1 AI辅助生产中的责任重新分配

在传统软件工程中，责任隐含地锚定于作者身份。

代码由可识别的个人或团队编写、审查和维护，使事后归因成为可能。

在AI辅助开发中，这种联系大大削弱：

- 生成路径是非确定性的，
- 代码作者身份在多个代理之间稀释，
- 实现经常被重新生成。

在这种条件下，责任不能再可靠地从过程参与中推断。

ODD通过将责任从作者身份重新分配到仲裁来解决这个问题。

ODD 中的责任不是源于谁写了代码，
而是源于谁在指定契约下接受了产出物。

这种转变在不需要透视AI内部的情况下保持了问责。

4.2 契约作为责任锚点

在ODD中，契约具有双重功能：

1. 约束定义

它们定义可接受的输出空间。

2. 责任锚定

它们建立明确的决策点，在这些点上发生接受或拒绝。

通过在契约边界而非持续监督时要求人类确认，ODD使责任：

- 明确，
- 可审计，
- 可扩展。

重要的是，ODD中的契约不被假定为完整的系统描述。

它们是控制工具，而非本体论模型。

4.3 超越正确性的验证

传统工作流中的验证主要关注相对于实现的正确性。

ODD扩展了验证的概念，包括：

- 契约合规性，
- 扰动下的鲁棒性，
- 对抗抵抗力。

这激发了一种分层验证结构，其中：

- 单元测试和集成测试评估功能行为，
- 变异测试评估测试充分性，
- 对抗测试探测边界条件和滥用场景。

关键是，验证针对产出物，而非实现。

实现可能变化；
被接受的产出物必须保持可辩护。

4.4 渐进信任和有条件的人类干预

ODD的核心设计原则是渐进信任。

ODD不是对所有任务应用统一的审查，而是根据以下因素动态调整验证严格程度：

- 估计的复杂性，
- 安全敏感性，
- 外部依赖。

人类干预仅在以下情况下触发：

- 契约模糊，
- 对抗代理不同意，
- 或检测到高风险条件。

这种方法在防止认知过载的同时保持人类权威。

人类注意力被视为稀缺的治理资源。

4.5 无需完全透明的治理

对AI辅助系统的常见反对意见是缺乏模型透明度。

ODD故意避免假设：

- 可解释的模型，
- 可解释的生成路径，
- 或确定性输出。

相反，它采用基于可观察结果和记录的接受决策的治理策略。

这使ODD与以下兼容：

- 专有模型，
- 演进的AI架构，
- 异构工具链。

治理不是通过内省实现的，而是通过受控接受。

4.6 风险、失败和明确的非目标

ODD不消除风险。

它使风险 *可读*。

ODD中的失败表现为：

- 契约违规，
- 验证阶段失败，
- 或未解决的仲裁。

这些失败模式是明确和可审计的。

同样重要的是ODD的非目标：

- 它不旨在自动化伦理判断。
- 它不保证最优实现。
- 它不替代领域专业知识。

ODD提供结构，而非全知。

4.7 对AI原生组织的影响

随着AI系统从助手变为主要生产者，组织面临结构性挑战：

当生产自动化时，如何维持合法性？

ODD提出了一个答案：

- 合法性源于受控接受，
- 权威通过仲裁行使，
- 责任通过明确契约保持。

这些影响超越软件工程，延伸到组织设计和治理，这在后续工作中进一步探讨。

4.8 过渡到局限性和未来工作

上述讨论的影响既表明了优势，也表明了固有的局限性。

下一节将考察：

- ODD不适用的场景，
- 潜在的失败模式，
- 以及经验验证的方向。

5. 局限性与未来工作

5.1 当前局限性

5.1.1 适用性边界

ODD最适合具有以下特征的软件开发场景：

- 可明确规格化的需求：需求可以被转化为精确契约
- 可验证的输出：产出物可以通过自动化测试验证
- 重复性任务：类似任务频繁出现，值得投资契约定义

ODD可能不太适合：

- 探索性编程：需求在开发过程中才逐渐明确
- 创意性工作：如游戏设计、艺术创作等主观性强的领域
- 一次性脚本：投资契约定义的成本超过收益

5.1.2 契约定义的挑战

契约定义本身是一项需要专业技能的工作：

- 契约过于宽松 → 无法有效约束AI输出
- 契约过于严格 → 限制了合理的实现变体
- 契约不完整 → 遗漏重要的边界条件

这需要"契约工程师"这一新角色的出现和成熟。

5.1.3 工具链成熟度

当前ODD工具链仍处于早期阶段：

- 契约语言和格式尚未标准化
- 变异测试工具的性能和覆盖率有待提升
- 与现有CI/CD管道的集成需要更多工作

5.2 未来工作方向

5.2.1 经验验证

本文建立了概念框架；经验验证是关键的下一步：

- 在真实项目中测量ODD的效果
- 比较ODD与传统方法的生产力和质量指标
- 识别ODD最有效的应用场景

5.2.2 工具开发

支持ODD的工具生态系统需要发展：

- 契约定义语言和IDE支持
- 自动化契约生成和建议
- 变异测试性能优化
- 产出物追溯和审计工具

5.2.3 组织采用研究

ODD的组织影响需要进一步研究：

- 角色转变和技能需求变化
- 团队结构和协作模式
- 培训和变革管理策略

5.2.4 与其他范式的集成

探索ODD与现有方法的协同：

- ODD + 敏捷：如何在迭代中应用契约
 - ODD + DevOps：如何集成到持续交付管道
 - ODD + 形式化方法：如何利用形式化验证增强契约
-

6. 结论

6.1 核心贡献回顾

本文提出了输出驱动开发（ODD），一种面向AI辅助软件工程的产出物中心方法论。

我们的核心贡献包括：

1. 概念重构：将软件工程的控制单元从代码转移到产出物
2. 责任重新分配：通过契约和仲裁机制重建AI时代的责任锚点
3. 验证扩展：超越正确性，包含契约合规性、鲁棒性和对抗抵抗力
4. 治理框架：在不依赖模型透明度的情况下实现可审计的控制

6.2 范式转变的意义

ODD代表的不仅是方法论创新，更是软件开发生产关系的系统性重构：

- 脑力劳动与执行劳动的分离：人类专注于"定义什么"，AI负责"如何实现"
- 从手工作坊到智能工厂：软件开发从依赖个人技艺转向依赖系统化流程
- 责任的制度化：从隐含的作者身份责任转向明确的契约责任

6.3 对从业者的建议

对于希望探索ODD的从业者，我们建议：

1. 从小处开始：选择一个边界清晰的模块尝试契约定义
2. 投资测试基础设施：变异测试是ODD的信任基础
3. 培养契约思维：学习如何将需求转化为可验证的规格
4. 保持人类判断：ODD不是完全自动化，人类仲裁仍然关键

6.4 展望

随着AI能力的持续提升，软件工程必将经历深刻变革。

ODD提供了一个框架，使这种变革可以在保持责任、可审计性和人类控制的前提下进行。

我们相信，产出物中心的思维方式将成为AI原生软件工程的基础范式之一。

*软件开发的未来不是人类与AI的竞争，而是人类智慧与AI能力的协同。
ODD为这种协同提供了结构性基础。*

参考文献

- [1] Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley.
- [2] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley.
- [3] Meyer, B. (1997). Object-Oriented Software Construction (2nd ed.). Prentice Hall.
- [4] Jia, Y., & Harman, M. (2011). An Analysis and Survey of the Development of Mutation Testing. IEEE Transactions on Software Engineering, 37(5), 649-678.
- [5] Stahl, T., & Völter, M. (2006). Model-Driven Software Development. Wiley.
- [6] Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- [7] OpenAI. (2023). GPT-4 Technical Report. arXiv:2303.08774.
- [8] White, J., et al. (2023). A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. arXiv:2302.11382.
- [9] Amershi, S., et al. (2019). Software Engineering for Machine Learning: A Case Study. ICSE-SEIP.
- [10] Sculley, D., et al. (2015). Hidden Technical Debt in Machine Learning Systems. NeurIPS.

附录A：ODD系列论文导航

本文是ODD系列的第一篇，建立了基础概念框架。系列其他论文包括：

论文	标题	核心主题
Paper I （本文）	AI原生软件工程中的产出物合法性基础	产出物、契约、责任锚点
Paper II	基于渐进式信任与对抗验证的人类可委托证明	变异测试、信任建立、对抗验证
Paper III	契约精准度作为AI代码生成质量的控制变量	契约规格、质量度量、精准度
Paper IV	AI原生软件组织的合法性演化与治理	组织变革、角色转型、治理
Paper S1	面向可审计LLM工作流的上下文工程	上下文管理、5D模型、工具支持

阅读建议：

- 理论研究者：Paper I → Paper II → Paper III → Paper IV
- 实践者：Paper I → Paper S1 → Paper II
- 管理者：Paper I → Paper IV

补充材料

注：以下内容为完整技术参考，保留自白皮书版本，供深入研究使用。

完整白皮书版本请参见：`Paper_01_ODD_Core_English_Whitepaper.md`